

Proyecto 2 – Exploradores del Laberinto

Curso: Taller de Programación

Grupo: 01

Estudiante: Fabián Gerardo Cambronero Núñez

Carné: 2026079420

Profesor: Diego Mora Rojas

Fecha de entrega: 09/06/26

Periodo: 1 Semestre 2026

Índice

1. Descripción general del problema	4
2. ¿Cómo se representa el mapa?	4
Ejemplo Mapa.....	5
Funcionamiento del mapa	5
3. Símbolos utilizados.....	5
Símbolos del mapa:.....	6
Símbolos del proceso de exploración	6
4. Formato de archivos	6
Archivos del mapa	6
Carga de mapas desde archivos	7
Guardado de mapas.....	7
Reportes de búsqueda	8
5. Interfaz.....	8
Ventana principal.....	8
Menú de botones principales	8
Área visual del mapa.....	8
Editor visual del mapa	9
Área de información del sistema.....	9
Ventanas emergentes	9
6. Algoritmo de Backtracking.....	9
Funcionamiento general del backtracking	10
Casos base del backtracking	10
Exploración de caminos.....	10
Marcado de casillas visitadas.....	11
Retroceso del algoritmo	11
Búsqueda del primer tesoro	11
Búsqueda de todos los tesoros	11
7. Capturas de pantalla de la aplicación	12
Captura cargando un mapa.....	13
Captura editando un mapa	13
Captura ejecutando la búsqueda.....	14

Captura mostrando el resultado	14
8. Mapas entregados	15
Mapa fácil	15
Mapa medio	15
Mapa difícil.....	16
9. Problemas encontrados	17
Conversión del clic del usuario a una celda de la matriz.....	17
Mantener una sola entrada en el mapa.....	17
Ciclos infinitos en el backtracking.....	17
Mostrar animación sin congelar la interfaz.....	17
10. Conclusiones	18

1. Descripción general del problema

El proyecto Exploradores del Laberinto Perdido consiste en el desarrollo de una aplicación gráfica que permite representar, crear, editar, cargar, guardar y resolver laberintos mediante el uso de matrices. El problema principal se basa en simular la exploración de un laberinto en el que existe una entrada, caminos libres, paredes, trampas, salidas y tesoros ocultos que deben ser encontrados por un explorador.

Cada laberinto se representa como un mapa formado por filas y columnas, donde cada casilla contiene un símbolo específico. Estos símbolos indican si una posición corresponde a una entrada, un camino libre, una pared, una trampa, un tesoro, una salida o una marca generada durante la búsqueda. El usuario puede interactuar con el mapa desde una interfaz gráfica, ya sea creando un laberinto desde cero, editando sus casillas manualmente o cargando un mapa previamente guardado en un archivo de texto.

El reto principal del proyecto es encontrar tesoros dentro del laberinto utilizando un algoritmo de backtracking recursivo. Este algoritmo debe iniciar desde la entrada del mapa y explorar los caminos posibles siguiendo un orden definido de movimiento: arriba, derecha, abajo e izquierda. Durante la exploración, el algoritmo debe evitar paredes, trampas, posiciones fuera del mapa y ciclos infinitos. Además, debe marcar visualmente las casillas visitadas, los retrocesos realizados y las rutas válidas hacia los tesoros encontrados.

El sistema permite realizar dos tipos de búsqueda: buscar el primer tesoro accesible y buscar todos los tesoros accesibles desde la entrada. En la primera búsqueda, el algoritmo se detiene cuando encuentra el primer tesoro. En la segunda, el algoritmo continúa explorando el mapa para determinar cuántos tesoros pueden alcanzarse y si existen tesoros inaccesibles debido a bloqueos, paredes o trampas.

Además de resolver el laberinto, el programa debe validar que los mapas cumplan las reglas establecidas, permitir guardar mapas en archivos de texto y generar reportes con el resultado de las búsquedas. De esta forma, el proyecto integra el uso de matrices, archivos, interfaz gráfica, validación de datos, recursividad y backtracking en una solución visual e interactiva.

2. ¿Cómo se representa el mapa?

El mapa del laberinto se representa internamente mediante una matriz de caracteres. En Python, esta matriz se implementa como una lista de listas, donde cada lista interna representa una fila del laberinto y cada elemento dentro de esa fila representa una celda específica del mapa.

Cada celda contiene un símbolo que indica el tipo de elemento ubicado en esa posición. Por ejemplo, la entrada se representa con “E”, los caminos libres con “.”, las paredes con “#”, los tesoros con “T”, las trampas con “X” y las salidas con “S”. Durante la ejecución del algoritmo de búsqueda también se utilizan marcas como V para casillas visitadas, R para retrocesos y * para rutas válidas encontradas.

Ejemplo Mapa

Un mapa guardado en archivo de texto puede tener la siguiente forma:

```
#####
#E..T#
#.#.#
#...#
#####
```

Al cargarse en el programa, cada línea del archivo se convierte en una fila de la matriz. Internamente, el mapa anterior se representa de forma similar a:

```
[
    ['#', '#', '#', '#', '#', '#'],
    ['#', 'E', '!', '!', 'T', '#'],
    ['#', '!', '#', '#', '!', '#'],
    ['#', '!', '!', '!', '!', '#'],
    ['#', '#', '#', '#', '#', '#']
]
```

Funcionamiento del mapa

Esta representación permite recorrer el laberinto usando índices de fila y columna. Por ejemplo, una posición puede identificarse como `mapa[fila][columna]`. Gracias a esto, el programa puede validar el contenido del mapa, editar celdas específicas desde la interfaz, cargar y guardar archivos, y aplicar el algoritmo de backtracking para explorar caminos y buscar tesoros.

La matriz también facilita verificar los límites del mapa, comprobar si una celda es transitable y registrar las marcas generadas durante la búsqueda. De esta manera, la representación por matriz cumple con el objetivo de organizar el laberinto de forma clara, manipulable y adecuada para aplicar recorridos recursivos.

3. Símbolos utilizados

El mapa del laberinto utiliza diferentes símbolos para representar los elementos que pueden aparecer en cada celda de la matriz. Cada símbolo tiene un significado específico y permite que el programa identifique si una posición puede recorrerse, si está bloqueada o si corresponde a un objetivo de búsqueda.

Símbolos del mapa:

Símbolo	Nombre	Descripción
E	Entrada	Representa el punto inicial desde donde comienza la exploración del laberinto. El mapa debe tener exactamente una entrada.
.	Camino libre	Representa una casilla transitable. El explorador puede moverse por estas posiciones.
#	Pared	Representa una casilla bloqueada. El explorador no puede atravesar paredes.
T	Tesoro	Representa un tesoro dentro del laberinto. Es el objetivo principal de la búsqueda.
X	Trampa	Representa una casilla peligrosa o bloqueada. El explorador no puede pasar por trampas.
S	Salida	Representa una salida del laberinto. En el programa se maneja como una casilla transitable opcional.

Además de los símbolos originales del mapa, el algoritmo de búsqueda utiliza marcas visuales para mostrar el proceso de exploración:

Símbolos del proceso de exploración

Símbolo	Nombre	Descripción
V	Visitado	Indica que el algoritmo pasó por esa casilla durante la exploración.
R	Retroceso	Indica que el algoritmo tuvo que retroceder desde esa casilla porque no encontró una ruta útil desde ese punto.
*	Ruta final	Indica que la casilla forma parte de una ruta válida hacia un tesoro encontrado.

Los símbolos E, ., #, T, X y S forman parte del mapa original creado o cargado por el usuario. En cambio, los símbolos V, R y * son generados durante la ejecución del algoritmo de backtracking para mostrar visualmente el recorrido, los retrocesos y las rutas encontradas.

Durante la búsqueda, el programa conserva los elementos importantes del mapa original. La entrada E y los tesoros T no son reemplazados visualmente por marcas de recorrido, y las paredes #, trampas X y salidas S tampoco se modifican. Esto permite que el usuario pueda observar el proceso de búsqueda sin perder la información original del laberinto.

4. Formato de archivos

El proyecto utiliza archivos de texto con extensión .txt para almacenar los mapas del laberinto y los reportes generados después de una búsqueda. El uso de archivos permite guardar mapas creados por el usuario, cargarlos nuevamente en el programa y conservar evidencia de los resultados obtenidos por el algoritmo de backtracking.

Archivos del mapa

Los mapas del laberinto se almacenan en archivos de texto .txt. Cada línea del archivo representa una fila del laberinto y cada carácter dentro de la línea representa una celda de la matriz.

El formato no utiliza comas, espacios, corchetes ni texto adicional. Esto permite que el archivo pueda convertirse directamente en una matriz de caracteres dentro del programa. Un ejemplo válido de archivo de mapa es el siguiente:

```
#####  
#E...#T#  
###..#.#  
#...##.#  
#.#X...#  
#.#####  
#T....S#  
#####
```

En este ejemplo, cada fila del archivo se convierte en una fila de la matriz. Por ejemplo, la línea #E...#T# se transforma internamente en una lista de caracteres donde cada símbolo ocupa una posición específica.

Carga de mapas desde archivos

La opción Cargar mapa permite seleccionar un archivo .txt desde una ventana de exploración de archivos. Al cargarlo, el programa lee el contenido línea por línea, elimina saltos de línea innecesarios y convierte cada línea en una lista de caracteres.

Después de convertir el archivo en matriz, el sistema valida que el mapa cumpla con las reglas establecidas. Entre las validaciones realizadas se encuentran: que el mapa no esté vacío, que todas las filas tengan la misma cantidad de columnas, que las dimensiones estén entre 5x5 y 20x20, que exista exactamente una entrada E, que exista al menos un tesoro T y que no haya símbolos inválidos.

Si el archivo cumple las reglas, el mapa se muestra gráficamente en la interfaz. Si el archivo contiene errores, el programa no lo carga como mapa activo y muestra un mensaje indicando el problema encontrado.

Guardado de mapas

La opción Guardar mapa permite almacenar el mapa actual en un archivo .txt. El archivo generado conserva el mismo formato utilizado para la carga: una fila por línea y un carácter por celda.

Antes de guardar, el programa crea una versión limpia del mapa. Esto significa que las marcas generadas durante una búsqueda, como V, R y *, se convierten nuevamente en caminos libres. De esta forma, el archivo guardado representa un mapa base y no un resultado de búsqueda.

Por ejemplo, si durante una búsqueda el mapa contiene marcas de recorrido, el botón Guardar mapa evita guardar esas marcas como parte del mapa original. Las marcas de búsqueda se reservan para los reportes de resultados.

Reportes de búsqueda

Además de guardar mapas, el programa permite guardar un reporte con el resultado de la última búsqueda ejecutada. Este reporte también se almacena como archivo .txt, pero su contenido es diferente al de un archivo de mapa, ya que incluye información descriptiva del proceso.

El reporte contiene el título del proyecto, la fecha y hora de generación, el tipo de búsqueda realizada, el mapa original, el mapa final después de la búsqueda, la cantidad total de tesoros, la cantidad de tesoros encontrados, las posiciones de los tesoros encontrados y el resultado final de la búsqueda.

Un reporte puede indicar, por ejemplo, si se encontró el primer tesoro accesible o si, al buscar todos los tesoros, existe al menos un tesoro inaccesible desde la entrada.

5. Interfaz

La aplicación Exploradores del Laberinto Perdido cuenta con una interfaz gráfica desarrollada en Tkinter. Su objetivo es permitir que el usuario interactúe con el laberinto de forma visual, sin necesidad de escribir comandos en consola. Desde la ventana principal se pueden crear mapas, cargarlos desde archivos, editarlos, guardarlos, ejecutar búsquedas y visualizar los resultados del algoritmo de backtracking.

Ventana principal

La ventana principal contiene todos los elementos necesarios para utilizar el sistema. En la parte superior se muestra el título del proyecto y un subtítulo relacionado con los temas principales utilizados: matrices, archivos, Tkinter y backtracking recursivo.

Esta ventana funciona como el contenedor general de la aplicación. Dentro de ella se organizan el menú de botones, el área visual del mapa y el área de información del sistema.

Menú de botones principales

Debajo del encabezado se encuentra el menú principal de acciones. Este menú contiene los botones necesarios para utilizar las funcionalidades del programa.

Área visual del mapa

El área visual del mapa es la sección principal de la interfaz. En esta zona se muestra el laberinto como una cuadrícula dibujada sobre un Canvas de Tkinter.

Cada celda de la matriz se representa gráficamente según su símbolo. Por ejemplo, la entrada se muestra con un diseño verde, las paredes con bloques oscuros, los caminos como suelo transitable, las trampas con color rojo, los tesoros como cofres y las salidas con color azul. Las marcas generadas por el algoritmo también se representan visualmente: las casillas visitadas, los retrocesos y las rutas finales se muestran con colores y símbolos distintos.

El tamaño de las celdas se calcula dinámicamente según las dimensiones del mapa. Esto permite que mapas pequeños y mapas de hasta 20x20 puedan visualizarse dentro del mismo espacio de trabajo.

Editor visual del mapa

La interfaz incluye un modo de edición que permite modificar el mapa directamente desde la ventana gráfica. Al presionar el botón Editar mapa, se abre una ventana con herramientas para seleccionar el tipo de celda que se desea colocar.

Las opciones de edición permiten colocar entrada, camino libre, pared, tesoro, trampa y salida. Después de seleccionar una herramienta, el usuario puede hacer clic sobre una celda del mapa para cambiar su contenido.

El sistema controla la regla de entrada única. Si el usuario coloca una nueva entrada E, la entrada anterior se elimina automáticamente, manteniendo solamente una entrada activa en el mapa.

Área de información del sistema

Debajo del mapa se encuentra el área de información del sistema. Esta sección muestra mensajes relacionados con el estado del programa, errores, confirmaciones y resultados.

Por ejemplo, en esta área se informa cuando un mapa se carga correctamente, cuando una búsqueda inicia, cuando se encuentra un tesoro, cuando existen errores de validación o cuando un resultado se guarda correctamente.

El área de información es de solo lectura para el usuario. El programa la actualiza automáticamente según la acción realizada.

Ventanas emergentes

El sistema utiliza ventanas emergentes para comunicar mensajes importantes. Estas ventanas se usan para errores, confirmaciones y avisos relevantes.

Por ejemplo, se muestran ventanas emergentes cuando un archivo cargado es inválido, cuando el mapa no cumple las reglas, cuando se encuentra un tesoro, cuando no existe ningún tesoro accesible, cuando se guarda un mapa, cuando se guarda un resultado o cuando el usuario intenta salir del programa.

6. Algoritmo de Backtracking

El algoritmo principal utilizado en el proyecto es backtracking recursivo. Esta técnica permite explorar diferentes caminos posibles dentro del laberinto, avanzar mientras existan casillas válidas disponibles y retroceder cuando se llega a un punto donde ya no es posible continuar.

En este proyecto, el backtracking se utiliza para simular el recorrido de un explorador dentro de una matriz. El explorador inicia desde la entrada E y se mueve únicamente por casillas transitables, como caminos libres ., tesoros T y salidas S. No puede atravesar paredes #, trampas X, posiciones fuera del mapa ni casillas que ya fueron visitadas.

Funcionamiento general del backtracking

El algoritmo funciona como una búsqueda recursiva. Cada llamada a la función representa la decisión de explorar una posición específica del mapa.

Cuando el algoritmo llega a una celda, primero verifica si esa posición es válida. Para que una posición sea válida debe estar dentro de los límites del mapa y debe corresponder a una casilla transitable. Si la posición no cumple estas condiciones, la función termina y retorna al punto anterior.

Si la celda es válida, el algoritmo la analiza y continúa intentando moverse hacia las posiciones vecinas. El orden de movimiento utilizado es el indicado por el proyecto:

1. Arriba
2. Derecha
3. Abajo
4. Izquierda

Este orden se mantiene igual durante toda la ejecución para que el comportamiento del algoritmo sea predecible y consistente.

Casos base del backtracking

Como el algoritmo es recursivo, necesita condiciones que indiquen cuándo debe detenerse una llamada. Estas condiciones se conocen como casos base.

En la búsqueda del primer tesoro, los principales casos base son:

1. Posición fuera del mapa: si la fila o columna no existe dentro de la matriz, la función retorna False.
2. Celda no transitable: si la celda es una pared, una trampa o una marca ya visitada, la función retorna False.
3. Tesoro encontrado: si la celda actual contiene un T, el algoritmo registra la posición del tesoro y retorna True.

El caso base exitoso ocurre cuando el algoritmo llega a una celda con tesoro. En ese momento no se usaron coordenadas previamente conocidas para llegar hasta él; el tesoro se detecta únicamente porque el recorrido llegó a una celda marcada con T.

Exploración de caminos

Después de validar una celda, el algoritmo intenta avanzar hacia las cuatro direcciones posibles. Para cada dirección calcula una nueva posición sumando o restando valores a la fila y columna actual.

Por ejemplo:

- Arriba: fila - 1, misma columna.
- Derecha: misma fila, columna + 1.

- Abajo: fila + 1, misma columna.
- Izquierda: misma fila, columna - 1.

Cada nueva posición se evalúa mediante una llamada recursiva. Esto significa que la función se llama a sí misma para intentar resolver el problema desde la nueva celda.

Marcado de casillas visitadas

Para evitar ciclos infinitos, el algoritmo registra las casillas que ya fueron exploradas. Esto es necesario porque un laberinto puede tener caminos conectados entre sí, y sin este control el explorador podría regresar constantemente a las mismas posiciones.

Durante la animación, las casillas recorridas se marcan con V, lo que indica que el algoritmo visitó esa posición. Una vez que una celda fue marcada como visitada, no se vuelve a considerar como una opción transitable en la misma búsqueda.

Este control permite que el algoritmo avance de forma segura y que cada camino sea evaluado una sola vez.

Retroceso del algoritmo

El retroceso es la característica principal del backtracking. Ocurre cuando el algoritmo llega a una celda desde la cual no puede continuar hacia ningún camino válido o no encuentra un tesoro por esa ruta.

Cuando esto sucede, la función recursiva termina y devuelve el control a la llamada anterior. En términos visuales, esa celda puede marcarse con R, indicando que el explorador tuvo que retroceder desde esa posición.

Este proceso se repite hasta que el algoritmo encuentra una ruta válida o hasta que confirma que no existen más caminos disponibles desde la entrada.

Búsqueda del primer tesoro

La opción Buscar primer tesoro ejecuta el backtracking desde la entrada E y se detiene cuando encuentra el primer tesoro accesible. Esta búsqueda no intenta localizar todos los tesoros del mapa, sino únicamente el primero que pueda alcanzarse siguiendo el orden de exploración establecido.

Cuando el algoritmo llega a una celda T, registra esa posición, finaliza la búsqueda y marca la ruta encontrada. Si después de explorar los caminos posibles no se encuentra ningún tesoro accesible, el programa muestra un mensaje indicando que no fue posible encontrar un tesoro desde la entrada.

Búsqueda de todos los tesoros

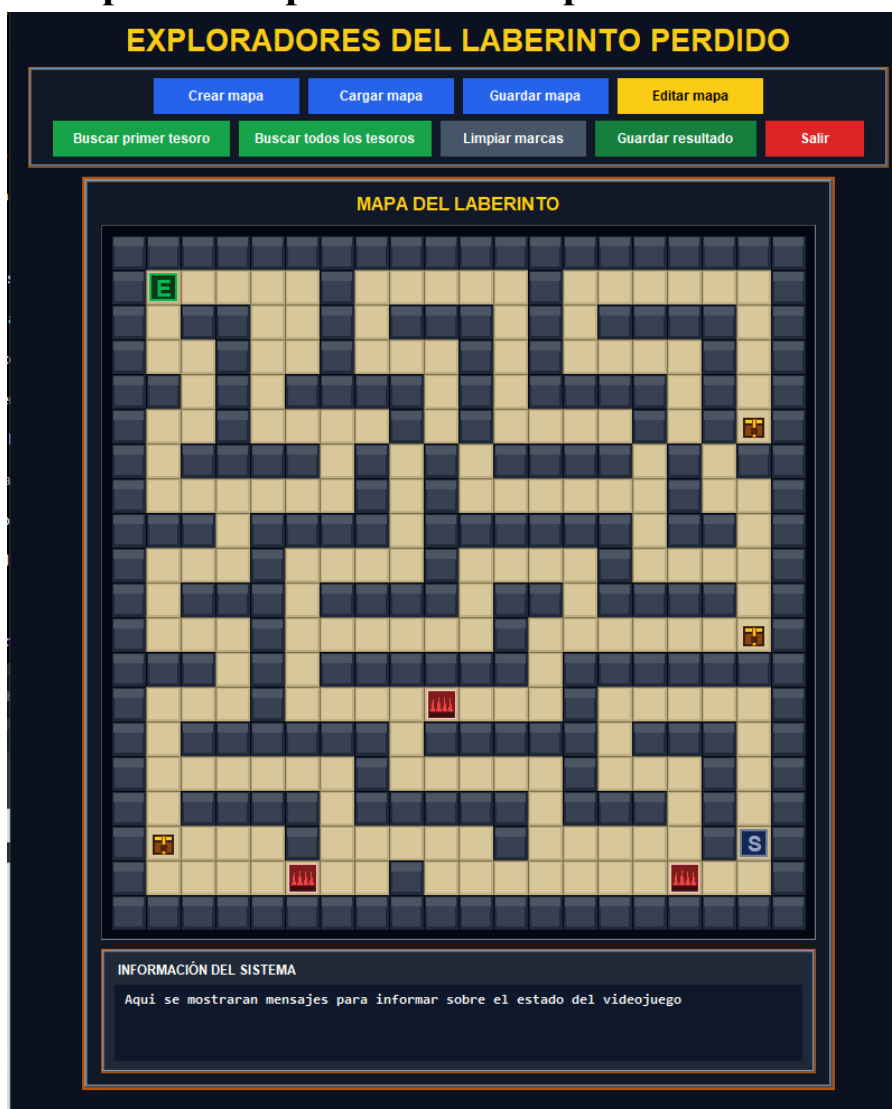
La opción Buscar todos los tesoros utiliza también backtracking, pero con una diferencia importante: el algoritmo no se detiene al encontrar el primer tesoro. En su lugar, continúa explorando todos los caminos accesibles desde la entrada.

Cada vez que el algoritmo llega a una celda T, registra la posición de ese tesoro como encontrado. Después continúa la exploración para verificar si existen más tesoros alcanzables.

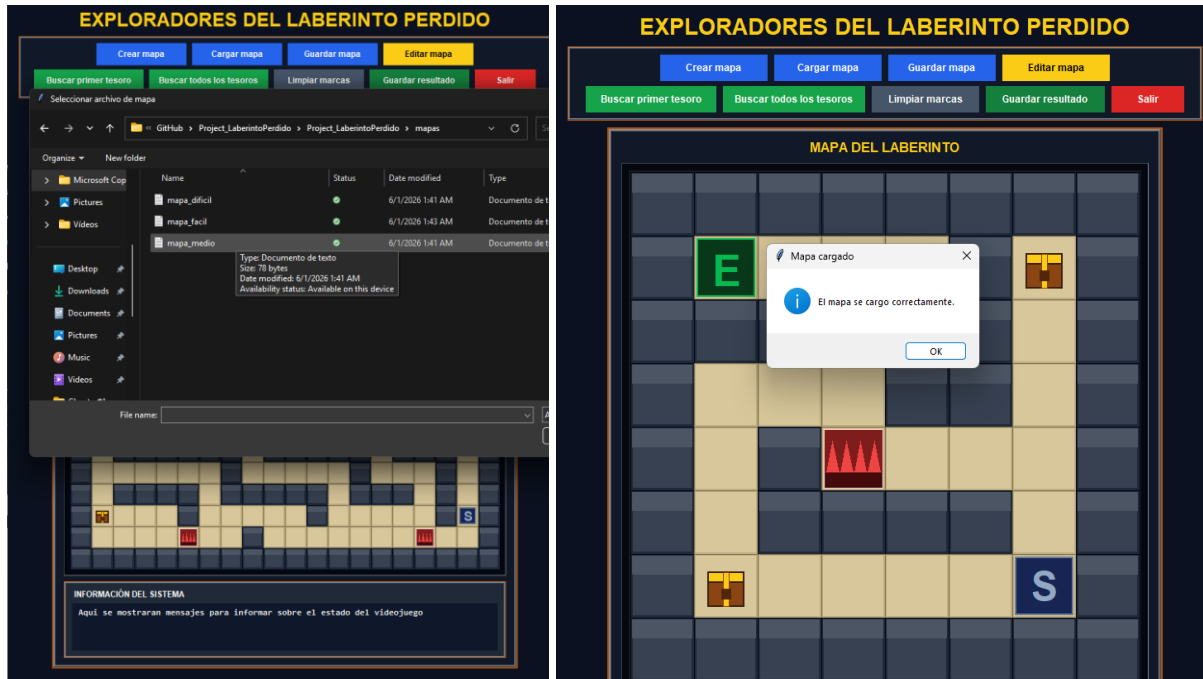
Antes de iniciar esta búsqueda, el programa cuenta cuántos tesoros existen en el mapa. Este conteo se utiliza únicamente para generar el resumen final; no se utiliza para guiar al algoritmo hacia posiciones específicas. Los tesoros se registran solamente cuando el recorrido llega a una celda T.

Al finalizar la búsqueda, el sistema compara la cantidad total de tesoros existentes con la cantidad de tesoros encontrados. Si ambos valores son iguales, significa que todos los tesoros son accesibles desde la entrada. Si la cantidad encontrada es menor que el total, significa que existe al menos un tesoro inaccesible, bloqueado por paredes, trampas o zonas no conectadas con la entrada.

7. Capturas de pantalla de la aplicación



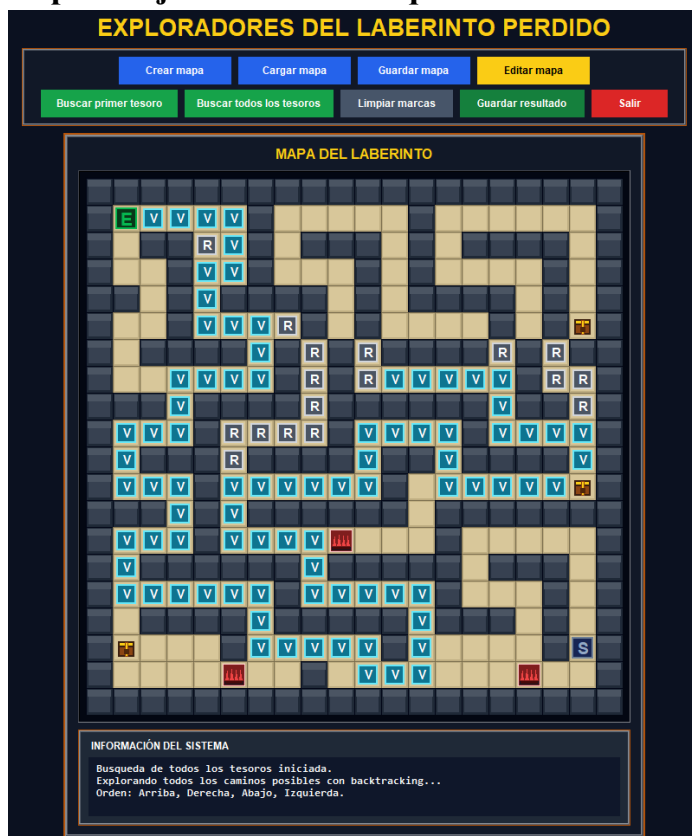
Captura cargando un mapa



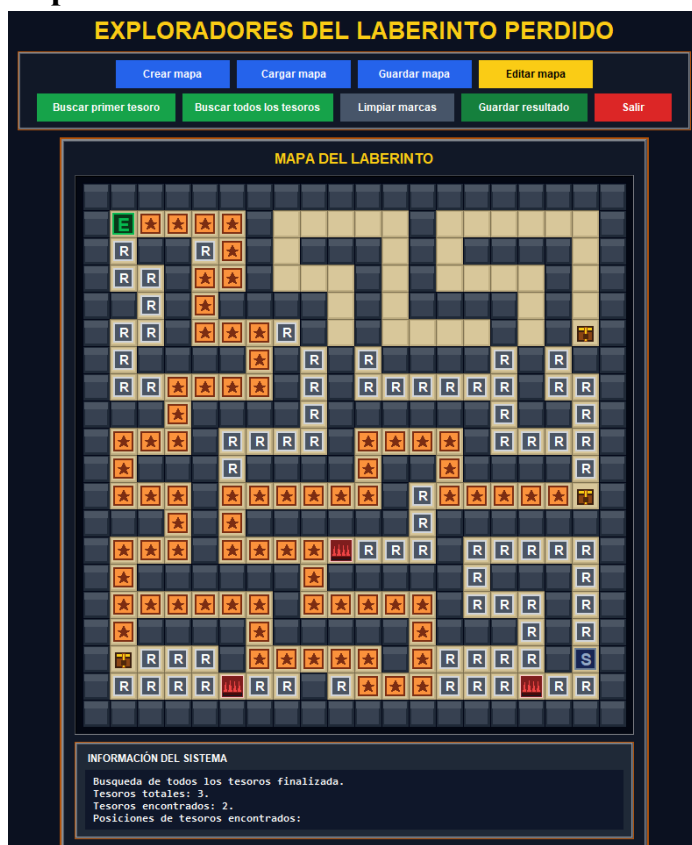
Captura editando un mapa



Captura ejecutando la búsqueda



Captura mostrando el resultado



8. Mapas entregados

Como parte de los entregables del proyecto, se incluyen tres mapas en archivos de texto dentro de la carpeta mapas. Estos mapas permiten probar diferentes niveles de dificultad y demostrar el funcionamiento de las principales características del programa, como la carga de archivos, la validación, la visualización gráfica, el backtracking, los retrocesos, la búsqueda de tesoros y la detección de tesoros inaccesibles.

Los mapas entregados son:

mapa_facil.txt

mapa_medio.txt

mapa_dificil.txt

Cada archivo cumple con el formato requerido: una fila por línea, un carácter por celda, sin comas, sin espacios, sin corchetes y utilizando únicamente los símbolos válidos del proyecto.

Mapa fácil

El archivo mapa_facil.txt contiene un laberinto pequeño y sencillo. Este mapa fue diseñado para comprobar el funcionamiento básico del programa y del algoritmo de búsqueda.

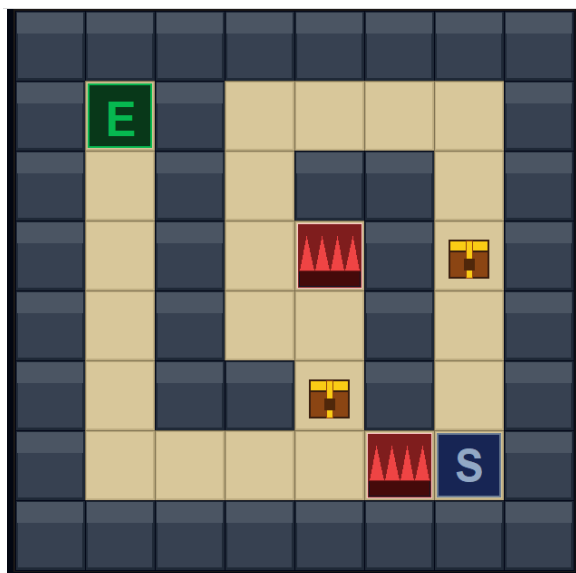
El mapa incluye una única entrada E, un tesoro T, caminos libres . y paredes #. contiene una única trampa, ya que su propósito principal es servir como una prueba inicial para verificar que el explorador puede desplazarse desde la entrada hasta un tesoro accesible. Al ser un mapa simple, es útil para validar rápidamente que la búsqueda del primer tesoro funciona de forma correcta.



Mapa medio

El archivo mapa_medio.txt presenta un laberinto con mayor dificultad que el mapa fácil. Incluye una entrada, al menos dos tesoros, paredes, caminos libres, una trampa y una salida.

Este mapa fue diseñado para obligar al algoritmo a tomar decisiones durante la exploración. Algunas rutas pueden llevar a caminos sin salida, lo que permite observar el comportamiento del backtracking cuando debe retroceder.

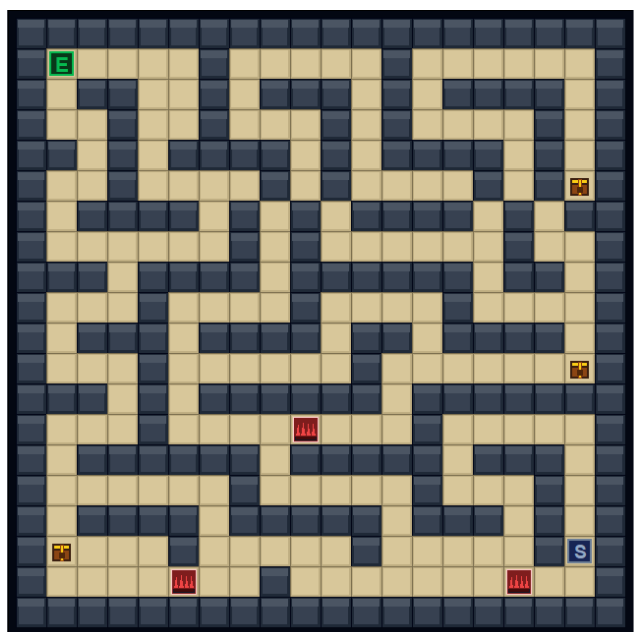


Mapa difícil

El archivo mapa_dificil.txt contiene el laberinto más complejo de los tres entregados. Este mapa tiene una entrada, varios tesoros, múltiples paredes, trampas, caminos sin salida y al menos un tesoro inaccesible.

Su objetivo principal es demostrar que el algoritmo no solamente encuentra tesoros accesibles, sino que también puede determinar cuándo no todos los tesoros pueden alcanzarse desde la entrada.

Durante la búsqueda de todos los tesoros, el programa explora todas las casillas accesibles desde E. Cada tesoro encontrado se registra únicamente cuando el algoritmo llega a una celda T. Al finalizar, el sistema compara la cantidad total de tesoros existentes en el mapa con la cantidad de tesoros encontrados. Si la cantidad encontrada es menor que el total, el programa reporta que existe al menos un tesoro inaccesible.



9. Problemas encontrados

Conversión del clic del usuario a una celda de la matriz

Uno de los primeros problemas fue lograr que el usuario pudiera editar el mapa haciendo clic sobre el área visual. El Canvas de Tkinter trabaja con coordenadas en píxeles, mientras que el mapa se maneja internamente como una matriz con filas y columnas.

Para resolverlo, se implementó una función que calcula el tamaño de cada celda, determina la posición inicial del mapa dentro del Canvas y convierte las coordenadas del clic en una posición de matriz. También se agregó una validación para ignorar los clics realizados fuera de la cuadrícula.

Mantener una sola entrada en el mapa

El proyecto indica que cada mapa debe tener exactamente una entrada E. Durante la edición visual, existía la posibilidad de que el usuario colocara más de una entrada.

Para solucionarlo, se implementó un control en el editor. Cuando el usuario coloca una nueva entrada, el programa busca si ya existe otra E en el mapa y la reemplaza por un camino libre. De esta manera, el mapa mantiene una única entrada de forma automática.

Ciclos infinitos en el backtracking

Al implementar el algoritmo de backtracking, uno de los principales retos fue evitar que el explorador regresara constantemente a las mismas casillas. Esto podía ocurrir porque los caminos del laberinto pueden estar conectados entre sí.

La solución fue marcar las casillas visitadas durante la búsqueda. En la búsqueda del primer tesoro se utilizan marcas dentro de la matriz, como V, para indicar que una celda ya fue explorada. En la búsqueda de todos los tesoros se utiliza además una matriz auxiliar de visitados. Esto permite que el algoritmo no repita posiciones y pueda terminar correctamente.

Mostrar animación sin congelar la interfaz

Otro problema importante fue visualizar el recorrido del algoritmo. Si el backtracking se dibujaba directamente mientras se ejecutaba, la ventana podía no actualizarse correctamente o mostrar todo el resultado de golpe.

Para resolverlo, el programa registra primero los pasos realizados por el algoritmo en una lista de animación. Luego, esos pasos se muestran progresivamente usando `ventana.after()`. Esto permite que la interfaz se actualice paso a paso y que el usuario pueda observar las casillas visitadas, los retrocesos y las rutas finales.

10. Conclusiones

El desarrollo del proyecto Exploradores del Laberinto Perdido permitió integrar varios conceptos fundamentales de programación en una sola aplicación funcional. Durante el proceso se trabajó con matrices, archivos de texto, validaciones, interfaz gráfica, eventos, recursividad y backtracking, logrando construir un sistema capaz de crear, editar, guardar, cargar y resolver laberintos de forma visual.

Una de las partes más importantes del proyecto fue la representación del laberinto como una matriz de caracteres. Esta estructura permitió manipular cada celda mediante filas y columnas, facilitando la edición del mapa, la validación de símbolos, la carga desde archivos .txt y la aplicación del algoritmo de búsqueda. Gracias a esta representación, el programa puede trabajar con diferentes mapas sin depender de rutas fijas o soluciones escritas manualmente.

El uso de archivos de texto fue fundamental para almacenar y recuperar mapas. El programa permite guardar mapas en un formato limpio y volver a cargarlos posteriormente. Además, se implementó el guardado de reportes de búsqueda, lo que permite conservar evidencia del mapa original, el mapa final y el resultado obtenido por el algoritmo.

La interfaz gráfica desarrollada con Tkinter permitió que el usuario interactuara con el sistema de manera clara y visual. A través de botones, ventanas emergentes, un editor de celdas y un área de mensajes, el usuario puede utilizar las funciones principales sin necesidad de ingresar comandos por consola. La visualización del mapa mediante colores y símbolos también facilita identificar los elementos del laberinto.

El algoritmo de backtracking fue el componente central del proyecto. Este permitió explorar los caminos posibles desde la entrada, evitar obstáculos, marcar casillas visitadas, realizar retrocesos y encontrar tesoros accesibles. Además, se implementaron dos modalidades de búsqueda: una para encontrar el primer tesoro y otra para encontrar todos los tesoros accesibles. La segunda modalidad permitió detectar cuando existen tesoros inaccesibles dentro del mapa.